Hackathon Technical Solution Report

Field	Details
Problem ID	`P2`
Problem Title	`Intelligent & Computationally Efficient Smartphone ISP (Hybrid RAW → sRGB pipeline)`
Team Name	`Rebels` (Team ID T42)
Team Members	Titiksha Surana – 1RUA24CSE0507 · Pranaith Rushi Raju – 1RUA24CSE0509 · V Spandana – 1RUA24CSE0510 · Vaibhav K – 1RUA24CSE0511
GitHub Repository	`https://github.com/spandana13-code/hybrid-isp` — evaluated commit `eaae99c`
Live Demo / Deployment	`https://huggingface.co/spaces/vspandana72/hybrid-isp` (web app on Hugging Face Spaces)
Demo Video	`https://github.com/spandana13-code/hybrid-isp/blob/main/docs/demo_video.mp4` (2-minute walkthrough of the live web app)

1. Problem Understanding & Solution Strategy

1. Problem Statement This problem involves converting a real Bayer RAW smartphone image into an accurate sRGB image. Due to the small sensor size on phones, high shot/read noise (ISO 50-10 000) is present in the image. There are multiple colour issues including inaccurate white balance (no metadata is provided in the official test set, therefore illuminant estimation is required). In addition, inaccurate colour matrix (a single DNG matrix causes a systematic bias in red) and demosaicing artefacts are present. The neural network must be lightweight, under 1MB, processing speed should be around 100-300ms per megapixel using 2 CPU threads. In addition, limited memory is available. Lastly, the quality must be consistent across 5 different phones with varying noise and brightness levels.
2. Solution The solution is divided into two parts based on the advantages of each approach. A classical pipeline is used for processing the inexpensive and well known steps including black level correction, CFA alignment, white balance, edge aware demosaic and calibrated colour matrix/gamma. The remaining steps where heuristics fail are processed by the 187k parameters learned network. An adaptive controller is used to calculate the noise profile and illuminant of each frame. This information is then used to condition the network (through the noise map), set the denoise and sharpening strength, and select the white balance source (metadata if the file has it, otherwise an estimate). This was selected over a fixed classical ISP due to a loss of around 3dB. It was also selected over an end to end neural network. Whilst the end to end network in FP32 provides equal PSNR to the hybrid model, it has lower color accuracy. Under INT8, it has lost 10x more quality. Also, it would need to be retrained for any new sensor.

2 Technical Architecture & Implementation

Architecture diagram: `docs/architecture.png`. Full design rationale: `docs/DESIGN.md`.

2.1 Classical Algorithmic Baseline

•Algorithms Used (isp/classical.py, isp/sidd.py):

Preprocessing:

•Bayer processing: black leveling, normalisation, lossless CFA conversion to RGGB (RGGB, GRBG and BGGR are used in the phones). Hot pixel clamping.

•Noise estimation:

•Variation in noise is estimated by fitting the model var(x) = ax + b using the difference between the two green channels (Gr-Gb)/[sqrt(2)]. This is done on flat areas. The MAD is calculated robustly.

•White balance:

Used the DNG AsShotNeutral value if available. Otherwise we estimate the white balance using shades-of-grey (Minkowski p=6). We exclude any clipped and dark pixels.

•Demosaicing:

Edge-aware interpolation (OpenCV EA).

•Colour Correction:

Using the DNG colour matrix ColourMatrix2 and a phone specific 3x3 colour correction matrix. This was calculated via least squares for training frames.

•Tone Mapping:

sRGB OETF. Auto exposure and a highlight knee are used for simulated inputs.

•Denoising:

Non-local means filter in CIELab. Chroma is filtered 1.5 times more than luma. Fast option uses bilateral filter in YCrCb.

•Sharpening:

Unsharp mask applied to luma only (YCrCb). Small details are soft-thresholded so noise is not amplified.

•Adaptive Selection Logic (analyze() in isp/classical.py): A single pass of the RAW provides the shot/read noise parameters (a,b), white balance gains, exposure gain and scene label. The controller then uses the noise model to estimate σ_out. This is used to determine the strength of the NLM filter (h $\propto$ σ out) and the amount of sharpening (0.7 − 6·σ out). For very noisy frames, sharpening is turned off. White balance source is selected automatically between metadata and estimation. These statistics are also used to form a pixel-wise noise map and thumbnail, to be fed as inputs into the hybrid network.

2.2 Deep Learning Pipeline

- **Model Architecture** (`isp/models.py`):

•LiteUNet - 3-stage UNet consisting of depth-wise separable residual blocks (24/48/96 channels). Approximately 8x less multiplies/adds than standard 3x3 convolutional blocks of the same width. Fits well into the mobile domain.

•LearnedISP - End-to-end deep learning baseline (PyNET / U-Net-ISP style; 115k parameters). Takes packed RGGB and noise map as input and learns to map directly to sRGB. It must learn the complex tasks of demosaicing, colour correction, tonemapping and denoising.

•HybridNet - Proposed network (187k parameters). Combines LiteUNet with additional global colour & exposure block (See Section 2.3).

Small custom CNNs were chosen due to the need for the network to be below 1MB, quantise to INT8 and run at interactive speeds on a CPU.

Loss Function and Training (train.py) L1 loss and 0.15 (1 - SSIM). L1 loss encourages pixel and colour accuracy. SSIM term maintains structure.

•AdamW optimiser was used with 1cycle learning rate schedule (max 2e-3). Batch size 16 and 4000 iterations. Best checkpoint selected based on PSNR during validation.

SIDD (Smartphone Image Denoising Dataset - Abdelhamed et al. CVPR 2018) dataset -

5 phones (Google Pixel, iPhone 7, Nexus 6, LG G4, Galaxy S6) of real noisy RAW images with clean sRGB ground truth.

101 training instances and 14 validation instances (6060/224 patches) stratified by phone. Official SIDD benchmark uses 928 blocks.

2.3 Hybrid Integration & Adaptive Mechanism

•Fusion Strategy:

The Fusion Strategy takes input data of classical front-end processing (white balanced, demosaicked, colour corrected, gamma encoded) plus per pixel noise map from controller's noise model. The input is PixelUnshuffled to half resolution. This results in a similar cost to the baseline model. The Fusion Strategy has two outputs:

Local residual (U-Net) - Denoising, demosaic artefact removal and detail retrieval. It uses the measured noise as conditioning. This only needs one network to handle all ISOs.

Global colour/exposure head - Takes a fixed size thumbnail (64x64) of the full frame. It produces a 3x3 matrix M = I + 0.5tanh(·) and log gain k. This is applied in linear space via y = OETF(M EOTF(x + r) e^k). This will correct any residual white balance and colour matrix issues. As it only outputs a matrix and gain it cannot hallucinate any content.

A final noise-gated luma sharpening stage is applied to keep it classical and controllable at runtime.

•Decision Logic:

Noise vs Detail:

The noise map is calculated per pixel based on the fitted shot/read model at the signal level of that pixel. This is then passed through the gain and gamma. This means regions that are dark have a higher value for sigma and so will be denoised more by the U-Net. While clean bright regions will retain their texture.

The sharpening after this has a factor of 0.5 times the amount of sharpening applied by the controller. This will reduce to 0 for very noisy frames meaning noise will never be amplified.

Gain:

For real phone RAW this will always be 1 as the exposure has been controlled by the phone. For under exposed simulated inputs, auto exposure will be applied by the controller using a highlight knee so bright regions won't be clipped and shadows will be lifted.

---

3 Experimental Benchmarks & Evaluation

All numbers come from the official SIDD benchmark. There are 928 256x256 real phone RAW image blocks (29 capture instances, 5 phones) not used in training, calibration or model selection. You can reproduce using python evaluate.py. Full tables can be found in eval_P2_T42.md. Per block results can be found in results/eval_rows.json.

3.1 Quantitative Benchmarks

Approach / Model Variant	PSNR (dB) ↑	SSIM ↑	Perceptual Metric (LPIPS) ↓	Latency (ms / frame, 1 MP)
Classical Baseline	26.25	0.7241	0.4475	536 ms
Deep Learning Only (INT8, deployable)	28.69	0.8637	0.2874	180 ms
Deep Learning Only (FP32)	*29.22*	*0.8828*	*0.2716*	*200 ms*
Proposed Hybrid System (INT8, deployed)	**29.12**	0.8647	0.2855	285 ms
Proposed Hybrid System (FP32)	*29.17*	*0.8675*	*0.2773*	*346 ms*

•Color fidelity (average CIEDE2000 ΔE ↓): 10.65 (classical), 8.83 (learned INT8), 7.86 (hybrid INT8, best). Latency refers to the total processing time of the full pipeline, including all classical stages and the network itself, on a real 1024×1024 SIDD frame using the ONNX Runtime on two CPU threads. The model sizes are 488 KB (hybrid INT8) and 202 KB (learned INT8).

- **Analysis:**
  -

Hybrid vs. classic: The hybrid outperforms the classical ISP with higher PSNR (+2.9dB), higher SSIM (+0.14) and lower LPIPS (-0.16). It also runs faster (1.9x speedup). With the learned denoiser replacing the slow NL-means (87% runtime of classical ISP).

Hybrid vs. pure learned: The hybrid and pure learned produce equivalent PSNR values when using FP32. The learned ISP produces a slightly smoother image (higher SSIM, lower LPIPS). However, the hybrid has the most accurate colours (ΔE 7.85 vs 8.07).

After quantising to INT8, the hybrid suffers only a small loss in PSNR (0.05dB) compared to the pure learned (0.53dB). This means that it provides the best results for deployable models with the highest PSNR (+0.43dB) and lowest ΔE (-0.97). This comes at the cost of an extra 100ms or so due to the slower classical front-end.

Robustness (PSNR, hybrid INT8 vs classical):

Condition	Classical	Hybrid INT8	Gain
Dark scenes	26.37	31.92	+5.6 dB
High noise	24.81	27.88	+3.1 dB
Bright scenes	26.88	27.02	+0.1 dB

The gain holds on every phone, from Google Pixel (24.88 → 26.99 dB) to Galaxy S6 (28.06 → 30.89 dB).

3.2 Visual Outputs & Intermediate Representations

1. Extreme low-light (see docs/comparison.jpg, row 1). Classical NLM produces noisy chromatic noise and green tint. These are removed in the hybrid version. See docs/intermediates.jpg, row 1. Input was near black and very noisy RAW image block.

2. Non-uniform / high brightness. Blocks used for testing in the SIDD benchmark are 256x256 crops without backlit scenes, so this is indicated by the brightness split results above, where high brightness blocks have no loss compared to classical method, and no clipped highlights. In docs/intermediates.jpg, row 3 we can see that we have retained highlights on the ColorChecker card (white text and dark body). Clipping has been managed by the knee in the controller using simulated inputs. The phone DNGs retain the exposure (gain=1) of the phone. All 58 test panels (block 0 and 13 of each test instance) can be found in:

   results/failure_cases/.

3. Intermediates. See docs/intermediates.jpg (generated by scripts/make_intermediates.py) to view three test blocks. Visualisation of Bayer RAW -> classical front-end output (input for the network) -> noise map (conditioning for the network, brighter = more noise, normalised for each row) -> learned residual (output of the network - input of the network, multiplied by 4. Mostly denoising in dark scenes and mostly edge/colour correction on the well lit card) -> hybrid output -> Ground Truth. The web app shows the noise σ, white balance source and gains, denoise h and sharpen applied at each step of the adaptive controller.
4. The website's "Walk the pipeline" view shows the image at every stage live (sensor mosaic, white balance, demosaic, colour matrix, tone curve, noise map, neural correction, output), along with the controller's decisions.

3.3 Ablation Study

•Each module was disabled in turn on the same 928 test blocks (`evaluate.py`; extra networks trained via `train.py --model hybrid_nonoise / hybrid_noglobal`):

Variant	PSNR	ΔE2000	Δ vs. full	Finding
Hybrid ISP (FP32), full	29.17	7.85	–	–
Hybrid, front-end only (no network)	23.93	13.69	−5.24 dB	the learned stage adds +5.2 dB / −5.8 ΔE over its classical input
Hybrid, no global colour head	29.02	8.26	−0.15 dB	the head is worth −0.41 ΔE and +0.027 SSIM
Hybrid, no noise-map conditioning	29.27	7.93	+0.10 dB	no measurable gain on SIDD; the network infers noise from the image

Hybrid, no post-sharpen	29.19	7.85	+0.02 dB	neutral
Classical, no white balance	19.81	19.86	−6.44 dB vs classical	white balance is the most important single stage
Classical, nominal instead of calibrated CCM	23.94	14.17	−2.31 dB	per-phone calibration is essential
Classical, no denoise	23.79	13.71	−2.46 dB	–
Classical, fixed (nonadaptive) parameters	26.41	10.62	+0.16 dB	on SIDD the adaptive NLM rule does not help; its strength mapping over-smooths (a tuning gap)
Classical, bilateral instead of NLM	27.35	9.99	+1.10 dB	same over-smoothing issue

Precision ablation: fully quantising the hybrid's 3×3 colour head to INT8 cost 0.45 dB. Mixed precision (U-Net INT8, head FP32) costs only 0.05 dB, which is why it is the deployed configuration.

4 Failure Modes & Limitations

•Failure Case 1: White-balance on monochrome scenes. 42 of 928 blocks had PSNR less than 18 dB in all pipelines. Worst is instance 0020 block 21 (Google Pixel), with hybrid PSNR of 10.72 dB and ΔE of 25.9. Scene has uniform saturated colour (orange cylinder/yellow surface) which resulted in a "beige" output for a 256x256 block (row 4 docs/comparison.jpg).

•Root Cause/Mitigation 1: As no metadata is present, we have to estimate the illuminant. Shades-of-grey makes the assumption that the minkowski mean of reflectance values in the scene will be achromatic (i.e. $\|\rho\|_p \propto (1,1,1)$). However, if the block is a single chromatic surface, the colours of the object are assumed to be due to the illuminant. Therefore, they are divided out. This is passed to all pipelines. We cannot overcome this issue using our learned global head since there are no neutral samples within the patch. To mitigate this:

•Use phone's as shot white balance when possible (already implemented for DNGs)

Estimate white balance from full frame

Train a learned illuminant estimator (like FC4/FFCC) using SIDD's AsShotNeutral labels.

If we exclude these 42 blocks, classical has 26.8 dB, learned has 29.9 dB and hybrid has 29.8 dB.

•Failure Case 2: Warm cast when using metadata white balance (discovered during testing). Sometimes when using the white balance information from the phone's DNG metadata, both networks will apply a warm cast to the image (approx +0.05 red). Can cause up to approx 5dB drop in performance on some test scenes. This is because half of the training crops were estimated. This means there is a bias in errors. The network could not determine which samples had reliable white balance, so ended up always applying an average correction. This can be overcome by adding a flag to the inputs indicating whether the white balance was estimated or from metadata. The head will only apply corrections if the white balance was not reliable. Training on crops between 256px and 768px would also fix the thumbnail scaling discrepancy. This fix has been designed but not yet trained. The benchmark numbers are unaffected as there is no metadata in the test set.

•Other Limitations:

Google Pixel was the most difficult phone to work with (27.0 dB). Captures were darker and noisier.

Hybrid had slightly more noise remaining in flat dark regions than the learned ISP (FP32 LPIPS 0.277 vs 0.272). This would be addressed by adding a perceptual loss term.

Short training time (101 captures, 4 000 iterations on CPU).

Metrics favour replicating SIDD's reference.

Latency was measured on x86 CPU to represent phone. No NPU was measured.

5 Deployment & Quickstart Guide

Quickstart Commands

```
        # 1. Clone the repository and install dependencies git
        clone https://github.com/spandana13-code/hybrid-isp cd
        hybrid-isp pip install -r requirements.txt

        # 2. Run inference on a sample directory (all 3 pipelines; PNGs + metrics.csv) python
        run_inference.py --input_dir ./examples/ --output_dir ./results/inference/ --
pipeline all

        # 3. Web app python app.py                    # →
        http://localhost:7860

        # 4. Reproduce the benchmark tables (needs the SIDD download, see README) python
        evaluate.py --team T42 --usns
"1RUA24CSE0510,1RUA24CSE0511,1RUA24CSE0509,1RUA24CSE0507"
```

Deployment Format

•Web app (app.py + web/index.html): a custom front-end with its own JSON API, served through Gradio. It has two views, Walk the pipeline and Compare results (with a four-way magnifier), plus score bars, the controller summary in plain language, the benchmark charts and an INT8/FP32 switch.

https://huggingface.co/spaces/vspandana72/hybrid-isp

The application is run using the same app.py script. Upon startup, the repository will be downloaded (launcher script can be found in hf_space/).

•Inference is performed using ONNX Runtime. The deployed models are INT8; FP16 and FP32 versions can also be selected. PyTorch is not required during inference.

The application accepts phone DNG/RAW images, SIDD .npz blocks (with ground truth for metrics) and JPG/PNG images (converted to simulated sensor RAW). The application shows every pipeline stage, the three pipeline outputs and the ground truth, the adaptive controller decisions and the metrics. It also includes loading and error screens.

Batch CLI also available (run_inference.py).

Appendix: Figures

Figure 1. System architecture (classical front-end, adaptive controller, learned stage, postprocessing).

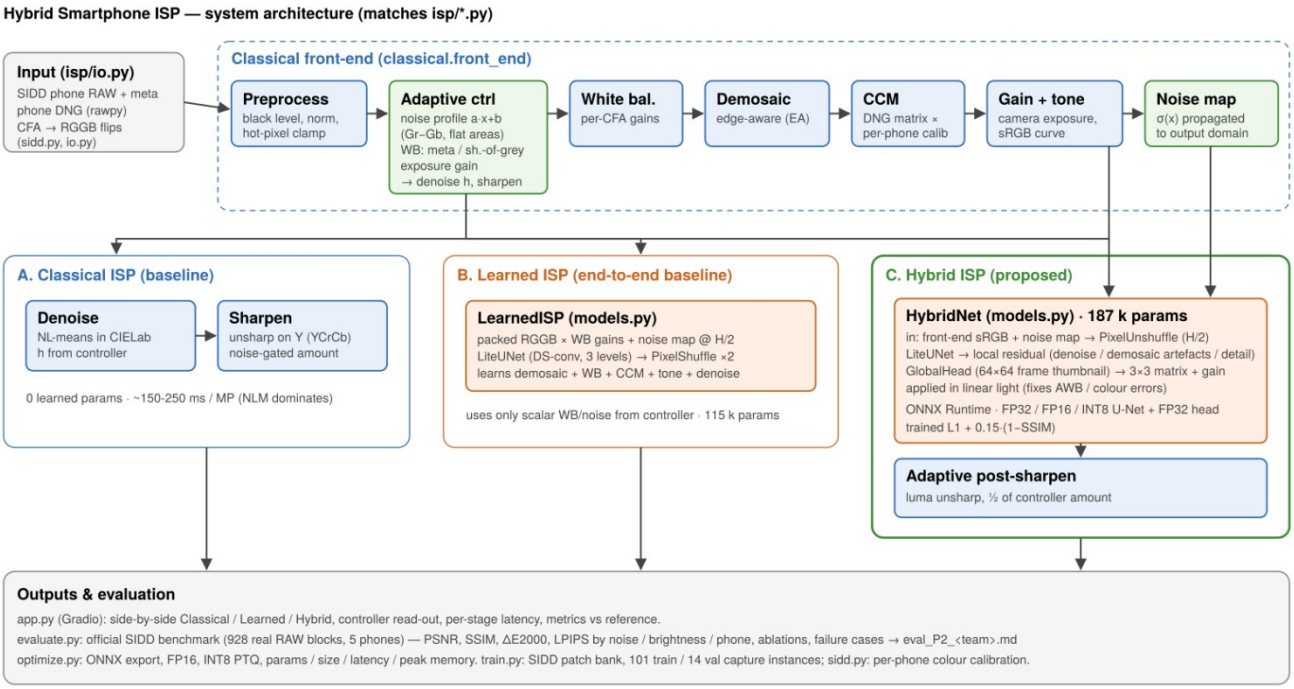


Figure 2. Official SIDD test blocks. Columns: ground truth | classical | learned | hybrid. Rows: dark fabric (high noise), flat card, text, and the main failure case (single-colour orange object, white balance).

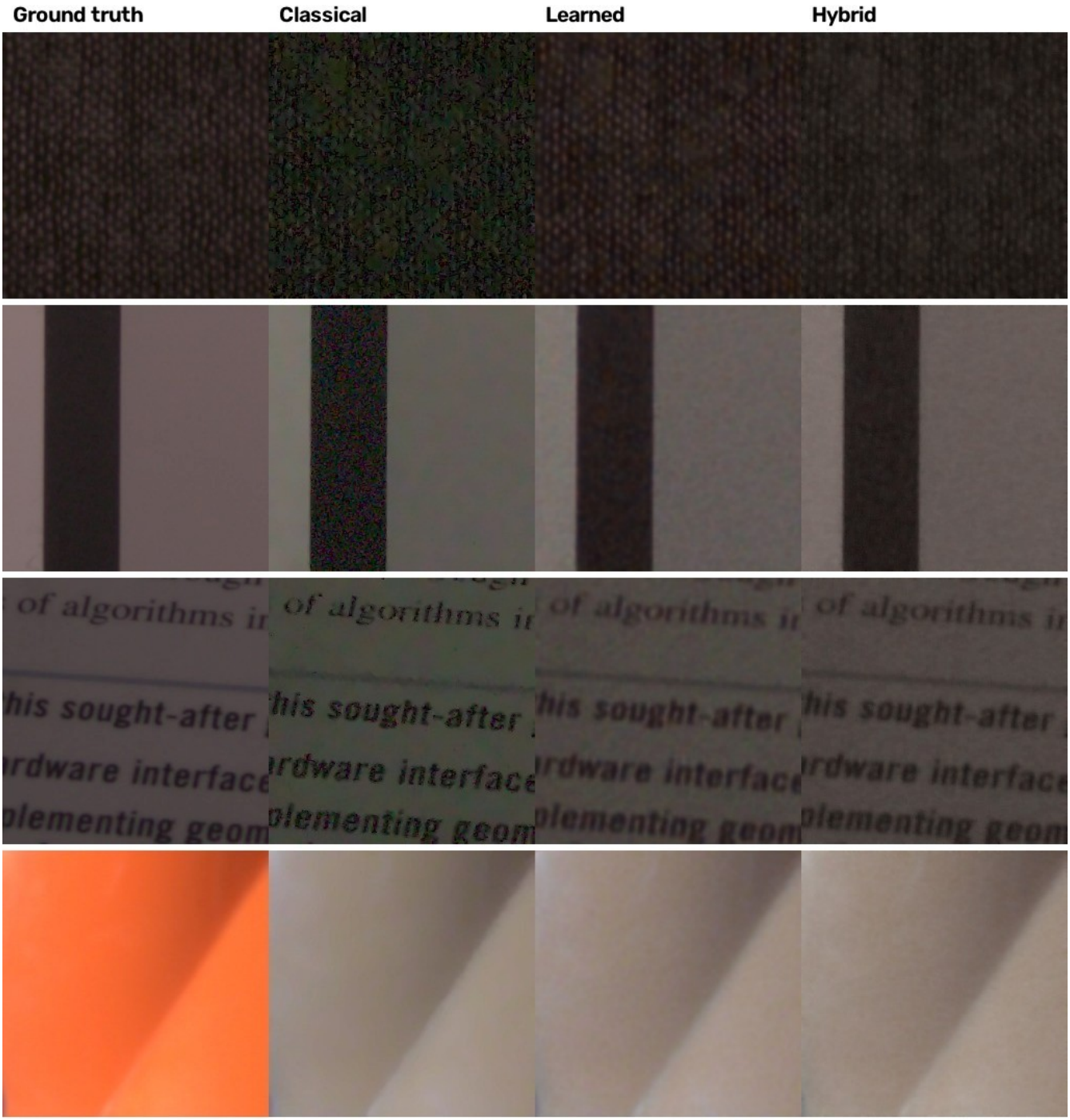
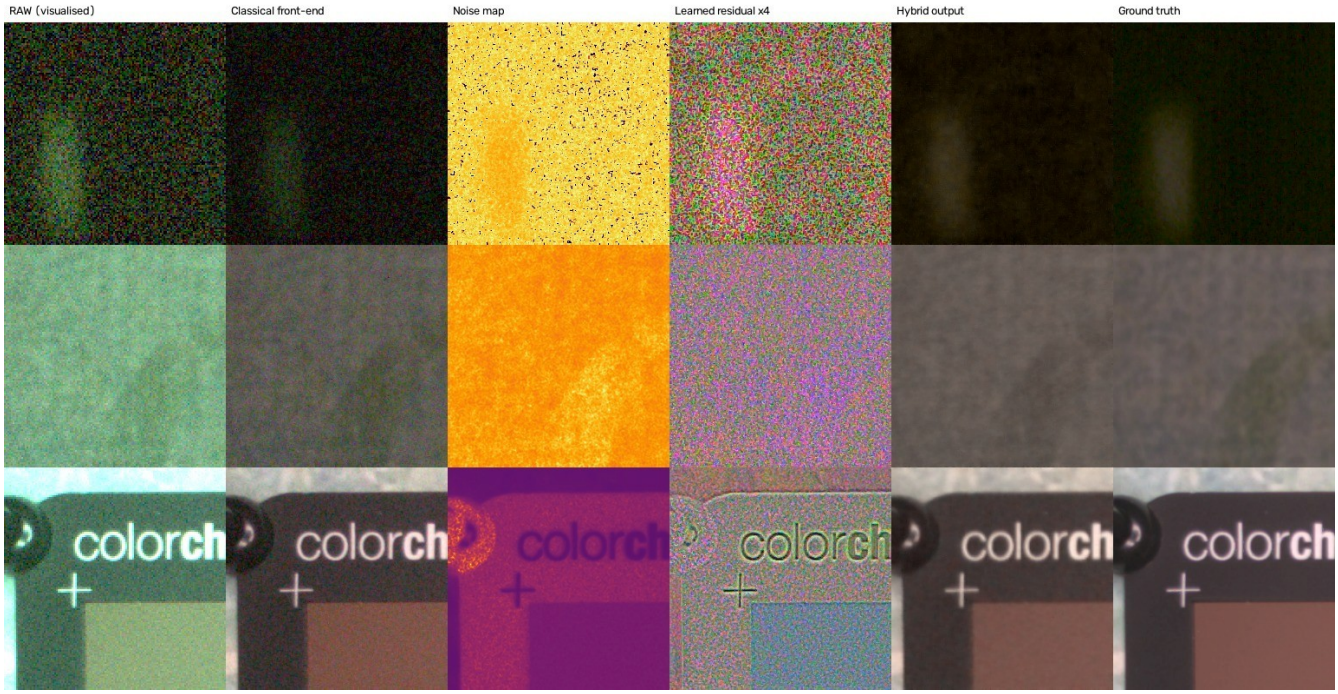


Figure 3. Intermediate representations of the hybrid pipeline (three SIDD test blocks). Columns: visualised RAW | classical front-end output (network input) | measured noise map | learned residual ×4 | hybrid output | ground truth.



StudentSubmissionChecklist

- ☑️ GitHub link to working repo provided with updated README.md file.
- ☑️ No code has been copied directly into this report.
- ☑️ All metrics shown in the tables can be reproduced by running the test scripts in the repo (evaluate.py > eval_P2_T42.md)
- ☑️ All team member information is correctly recorded.